

Git & GitHub — Complete Notes (Roman Urdu)

1. Version Control System (VCS) Kya Hota Hai?

- **Git** ek **Version Control System** hai jo code ki history track karta hai.
- Bilkul bank statement ki tarah — jaise bank account mein credit/debit ki poori history hoti hai, waise hi Git humare project ki poori history rakhta hai:
 - Kab nayi file add hui
 - Kab file delete hui
 - Kis line mein kya change/update hua

Git kyun use karte hain?

1. **Sabse popular** VCS duniya mein hai (companies bhi isi ko use karti hain).
2. **Free aur Open Source** hai.
3. **Fast aur Scalable** hai — chahe chota project ho ya bada.

Git ke 2 Primary Kaam

1. **Track History** — project ki har change ki history save rehti hai, hum kabhi bhi purani state pe wapas ja sakte hain.
2. **Collaborate** — team ke saath kaam karna. Multiple developers ek project pe kaam karte hain aur Git track karta hai kis developer ne kya change kiya, taake ek dusre ke changes overwrite na hon.

2. GitHub Kya Hai?

- **Git** ek tool/software hai jo humare computer system pe chalta hai.

- **GitHub** ek website hai (github.com) jahan developers apna code store aur manage karte hain.
- Project folders ko GitHub ki language mein **Repository (Repo)** kehte hain.
- Jaise Instagram profile pe photos upload karte hain, waise hi developers apni GitHub profile pe apne projects (repos) upload karte hain — resume/portfolio ke saath GitHub link dete hain taake HR/interviewer verify kar sake.
- GitHub pe dusron ki repos bhi dekh sakte hain, unki copy (fork) bana sakte hain, aur usme khud ke changes daal sakte hain.

GitHub Account Setup

- Email address chahiye (personal Gmail ya college email — personal email better hai kyunki college email graduate hone ke baad kaam nahi karega).
- Sign Up → Email → Password → Username → Verify → Create Account.
- Naya repo banane ke liye: **Repositories** → **New** → Name do → Description (optional) → Public/Private choose karo → "Add a README" tick karo → Create.
- **README.md** ek special file hai jisme project ki details likhi jati hain (kya hai, kaise use karna hai, features, etc.) — ye **Markdown** file hoti hai.

Public vs Private Repo

- **Private:** sirf humein dikhti hai (Instagram private story jaisa).
- **Public:** sabko dikhti hai (Instagram public account jaisa).

Commit Kya Hota Hai (GitHub pe)

- Kisi change ko **finalize** karna commit kehlata hai — us change ka "screenshot" le lena.
- Rishta/relationship jaisa: pehle **engagement** (Git mein: **Add**), phir **wedding/commitment** (Git mein: **Commit**).
- GitHub (website) pe Add wala step skip ho jata hai, seedha commit hota hai.

- Har commit ke saath ek **commit message** likhte hain jo batata hai ke kya change hua ya kyun hua.
-

3. Git Ko System Pe Setup Karna

Zaroori Tools

1. **VS Code (Visual Studio Code)** — free, open-source, popular code editor (Microsoft ne banaya).
2. **Windows:** Git Bash download karna hoga.
3. **Mac:** Terminal already installed hota hai (Finder → search "Terminal").

Verify Karna

```
git --version
```

Agar version print ho jaye to matlab Git already installed hai.

Git Config (Setup)

Git ko batana padta hai ke changes kis account/email se associate karne hain:

```
git config --global user.name "apna username"  
git config --global user.email "apna-email@example.com"
```

- **Global** config = poore system ke liye ek hi account.
- **Local** config = ek specific repo/project ke liye alag account.

Config check karne ke liye:

```
git config --list
```

4. Basic Git Commands

`git clone` — Repo ko Local Machine Pe Lana

- **Remote** = GitHub pe jo files/folders hain.
- **Local** = apna laptop/computer.
- Clone kisi remote repo ki **copy/duplicate** local system pe banata hai.

```
git clone <repo-link>
```

- GitHub pe repo ke "Code" button se **HTTPS link** copy karo (beginners ke liye HTTPS recommended hai, SSH bhi option hota hai).

`cd` — Change Directory

```
cd folder-name
```

Folder ke andar jaane ke liye. Bahar aane ke liye:

```
cd ..
```

`ls` — List Files

```
ls          # normal files/folders list karta hai  
ls -a       # hidden files bhi dikhata hai (jaise .git folder)
```

- Har Git-tracked folder mein ek hidden `.git` folder hota hai — ye batata hai ke ye folder Git repo hai.

`git status` — Status Check Karna

```
git status
```

Batata hai:

- Kaunsi branch pe hain
- Kya sab kuch up to date hai
- Kaunsi files modified/untracked/staged hain

5. File States (4 Types)

State	Matlab
Untracked	Nayi file jiske baare mein Git ko pata hi nahi (kabhi commit nahi hui)
Modified	File mein change kiya gaya hai
Staged	File <code>git add</code> ho chuki hai, ab commit ke liye ready hai
Unmodified/Committed	Koi bhi pending change nahi, sab kuch committed hai

Flow: Untracked/Modified → (git add) → Staged → (git commit) → Unmodified

6. `git add` aur `git commit`

`git add` — Staging Area Mein Lana

```
git add filename.html      # ek specific file add karna
git add .                  # saari files ek saath add karna
```

- Ye new/changed files ko **Staging Area** mein le jata hai.
- Staging Area = "engagement" wali stage, jo Modified/Untracked aur Commit ke beech mein hoti hai.

`git commit` — Change Ko Record Karna

```
git commit -m "meaningful message"
```

- Commit = record of change (jaise "wedding" — legally record ho jata hai).
 - Commit message meaningful hona chahiye, jaise: `Add new button`, `Fix login bug`.
-

7. `git push` — Local Se Remote Pe Bhejna

```
git push origin main
```

- **origin** = GitHub repo (remote) ka default naam jahan se humne clone kiya tha.
- **main** = branch ka naam (pehle "master" hota tha, ab convention change ho gaya hai aur default branch "main" hoti hai).
- Pehli baar push karte waqt VS Code permission maangega — GitHub ko authorize karna padta hai.

Shortcut: `-u` flag

```
git push -u origin main
```

- `-u` = **upstream set** karna. Ek baar set karne ke baad future mein sirf `git push` likhne se kaam chal jayega.
-

8. `git init` — Naya Local Repo Banana

Jab hum khud apne computer pe naya project shuru karte hain (GitHub pe pehle se bana ke clone nahi karte), tab:

```
git init
```

- Ye current folder ko Git repository bana deta hai (`.git` folder create hota hai).
- Isko "Initialized empty Git repository" message confirm karta hai.

Naye Local Project Ko GitHub Se Connect Karna

1. GitHub pe naya repo banao (bina README ke, kyunki wo conflict create karega).
2. Local repo ko remote se link karo:

```
git remote add origin <repo-link>
```

3. Verify karo remote set hua ya nahi:

```
git remote -v
```

4. Push karo:

```
git push -u origin main
```

9. Git Workflow (Overall Summary)

1. GitHub pe repo banao (ya local pe `git init` karo).
 2. Repo ko clone karo (agar GitHub se start kiya).
 3. Changes karo files mein.
 4. `git add` — changes ko stage karo.
 5. `git commit -m "message"` — changes ko commit karo.
 6. `git push` — changes ko GitHub pe bhejo.
-

10. Git Branches

Branch Kya Hoti Hai?

- Jaise ek project pe multiple teams/developers alag-alag features pe kaam karte hain, har koi apni **alag copy (branch)** banata hai.
- Isse har developer ko doosre ka kaam khatam hone ka wait nahi karna padta — sab apni branch pe independently kaam karte hain, baad mein **main** branch ke saath **merge** kar dete hain.

Branch Commands

```
git branch                                # current branch dekhna / saari branches list  
karna
```



```
git branch -m new-name      # current branch ka naam change karna
git checkout branch-name    # doosri branch pe switch karna
git checkout -b new-branch   # nayi branch banana aur usme switch karna
git branch -d branch-name    # branch delete karna (jis branch pe khade ho, wo
                             delete nahi ho sakti)
```

Note: Pehle default branch "master" hoti thi, ab "main" hoti hai (kyunki "master" term ko English mein derogatory maana jata hai).

11. Branches Ko Push Karna

```
git push origin feature-branch-name
```

- GitHub automatically wo branch create kar dega remote pe agar wo already nahi hai.
-

12. Merging (Do Branches Ko Combine Karna)

Merge = do branches ke code ko same/match kar dena.

2 Tareeke Merge Karne Ke:

1. Command Line Se (`git merge`)

```
git diff branch-name      # do branches ke differences dekhna
git merge branch-name      # branches ko merge karna
```

2. Pull Request (PR) Se (GitHub Pe)

- **Pull Request** = GitHub ko batana ke "meri branch ke changes main branch ke saath merge karo".
- Bade projects mein senior developer/project manager PR ko **review** karta hai, comment karta hai, phir approve/merge karta hai.
- Steps: GitHub pe "Compare & pull request" → message likho → Create pull request → Merge pull request → Confirm merge.

`git pull` — Remote Se Changes Lokal Mein Lana

```
git pull origin main
```

- GitHub pe merge hone ke baad, wo changes local system mein bhi lana padta hai — iske liye `git pull` use hota hai.
 - Pull = remote repo se content **fetch aur download** karke local repo mein le aana.
-

13. Merge Conflicts

- **Merge Conflict** tab hota hai jab do branches ne **same file ki same line** mein alag-alag changes kiye hon, aur Git decide nahi kar pata konsa change rakhna hai.
- VS Code conflict lines ko highlight karta hai (`<<<<<<<`, `=====`, `>>>>>>>`) jaise markers ke saath) aur options deta hai:
 - **Accept Current Change** — jo abhi ki branch mein hai
 - **Accept Incoming Change** — jo doosri branch se aa raha hai
 - Ya dono changes manually rakh sakte hain

Resolve Karne Ka Process:

1. Conflict markers wali extra lines delete karo.

2. Decide karo kaunsa/kaunse changes rakhne hain.
 3. File save karo.
 4. `git add` aur `git commit -m "message"` karo.
 5. Phir push karo.
-

14. Undoing Changes (Changes Wapas Lena)

Case 1: Staged Changes Ko Unstage Karna (Add hua, commit nahi hua)

```
git reset filename      # ek specific file reset karna
git reset               # saari staged files reset karna
```

Case 2: Ek Commit Ko Undo Karna

```
git reset HEAD~1
```

- **HEAD** = sabse latest commit (jiska abhi screenshot liya gaya).
- `HEAD~1` = ek commit peeche jaana.

Case 3: Multiple Commits Peeche Jaana (Specific Commit Tak)

```
git log                # saare commits (aur unke hash) dekhna
git reset <commit-hash>
```

- Har commit ka apna unique **hash** hota hai — us commit ka hash copy karke reset kar sakte hain.

--hard Flag

```
git reset --hard <commit-hash>
```

- Isse changes Git se **aur** VS Code (working files) dono se hat jate hain (complete rollback).
-

15. Fork Karna

- **Fork** = kisi doosre ke project ki **rough copy** apne account mein banana.
 - Use case: Open source contribute karna, kisi company ki repo pe kaam karna, dost ke project pe collaborate karna.
 - GitHub pe repo dhoondo → **Fork** button pe click karo → apne account mein copy ban jayegi.
 - Fork ke baad wahan changes/improvements karke original repo ke owner ko **Pull Request** bhej sakte hain (agar wo permission de).
 - **Important:** Fizzool/unnecessary pull requests kabhi create nahi karni chahiye — sirf jab genuinely useful change ho.
-

16. Quick Command Reference (Cheat Sheet)

Command	Kaam
<code>git --version</code>	Git version check karna
<code>git config --global user.name "name"</code>	Username set karna
<code>git config --global user.email "email"</code>	Email set karna

Command	Kaam
<code>git config --list</code>	Config settings dekhna
<code>git clone <link></code>	Repo ko local mein clone karna
<code>git status</code>	Current status check karna
<code>git add filename</code>	Specific file stage karna
<code>git add .</code>	Saari files stage karna
<code>git commit -m "msg"</code>	Changes commit karna
<code>git push origin main</code>	Changes GitHub pe bhejna
<code>git push -u origin main</code>	Push + upstream set karna
<code>git init</code>	Naya local Git repo banana
<code>git remote add origin <link></code>	Remote repo se connect karna
<code>git remote -v</code>	Remote verify karna
<code>git branch</code>	Branches dekhna
<code>git branch -m new-name</code>	Branch rename karna
<code>git checkout branch-name</code>	Branch switch karna
<code>git checkout -b new-branch</code>	Nayi branch banana
<code>git branch -d branch-name</code>	Branch delete karna
<code>git diff branch-name</code>	Differences dekhna

Command	Kaam
<code>git merge branch-name</code>	Branches merge karna
<code>git pull origin main</code>	Remote se changes lana
<code>git reset filename</code>	File ko unstage karna
<code>git reset HEAD~1</code>	1 commit peeche jaana
<code>git reset <hash></code>	Specific commit tak jaana
<code>git reset --hard <hash></code>	Complete rollback (Git + working files)
<code>git log</code>	Commit history dekhna

17. Important Concepts Summary

- **Repository (Repo):** Project folder jisme code hota hai.
- **Origin:** Remote repo (GitHub) ka default naam.
- **Main/Master:** Default branch ka naam.
- **HEAD:** Sabse latest commit ka pointer.
- **Staging Area:** Add aur Commit ke beech ki state.
- **.git folder:** Hidden folder jo batata hai ke ye directory Git-tracked hai.